



RouteGuard

Autonomous retail supply chain recovery

Gaurav Vyas · Team leader
Utkarsh Umang · Shivam Salve



The recovery problem

A late shipment threatens store inventory; the recovery must respect every constraint.

100 units

needed before the deadline

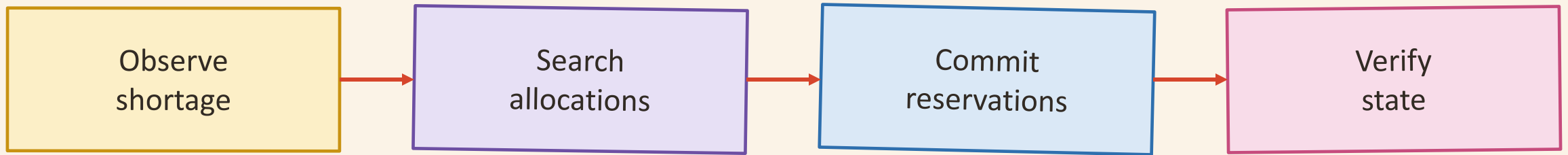
INR 14,000 budget
24-hour deadline
110 kg CO2e limit

Target users: retail operations teams coordinating inventory, suppliers and transport.

Simulation: 20 units on hand; demand 120; original shipment ETA 36 hours.

A goal-driven recovery loop

Actions change the environment; observations determine the next action.

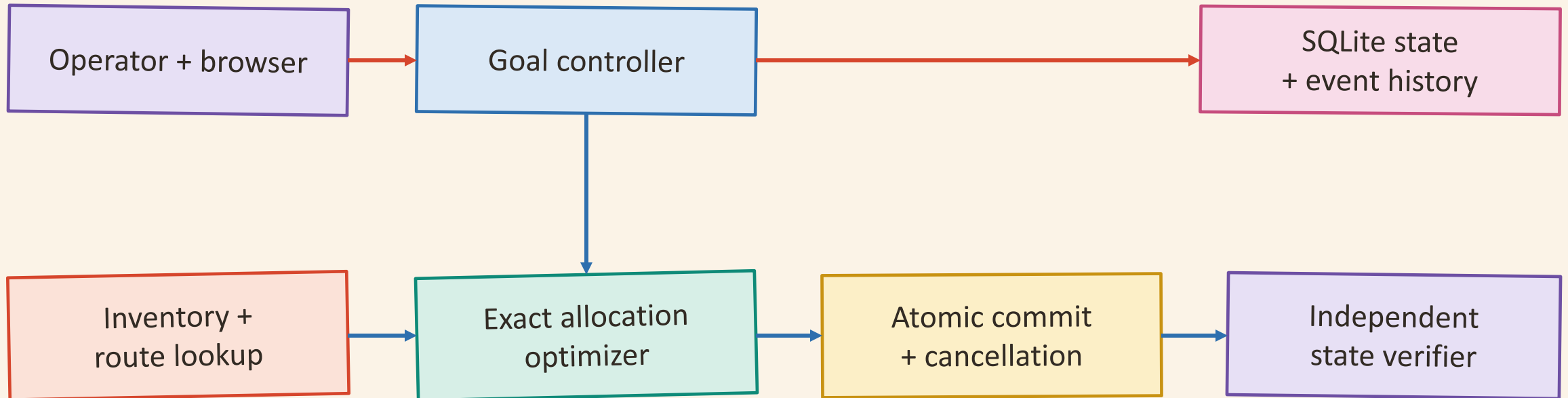


Tool rejection or changed conditions → refresh state → replan

Persistent memory: goal, catalog revision, orders, ledger, receipts and event history.

System architecture

Local tools and persistent state; no credentials or internet needed to run.



Failure feedback returns to the controller; unresolved cases go to operator review.

Two disruptions, three computed plans

The agent preserves valid reservations and replans only the remaining shortage.

Initial plan

Cuttack: 100 units

INR 9,000 · Rejected: supplier withdrawn

First recovery

Puri: 60 + Khordha: 40

INR 10,600 · Committed, then Puri closes

Final recovery

Khordha: 80 + Berhampur: 20

INR 11,700 · All constraints verified

Verified recovery

Independent checks use saved orders, inventory and ledger, rather than cached planner totals.

100

recovery units

INR 11,700

38 kg CO₂e / latest ETA 22 h

10 checks passed

Quantity · cost · carbon · deadline · routes · ledger · order IDs · stock · prices · receipts

Reproducible evaluation

80 synthetic cases, seed 1742838, checked against an independent dynamic-programming oracle.

80 / 80

31 / 31 feasible cases verified
49 / 49 infeasible cases identified
17 automated tests passing

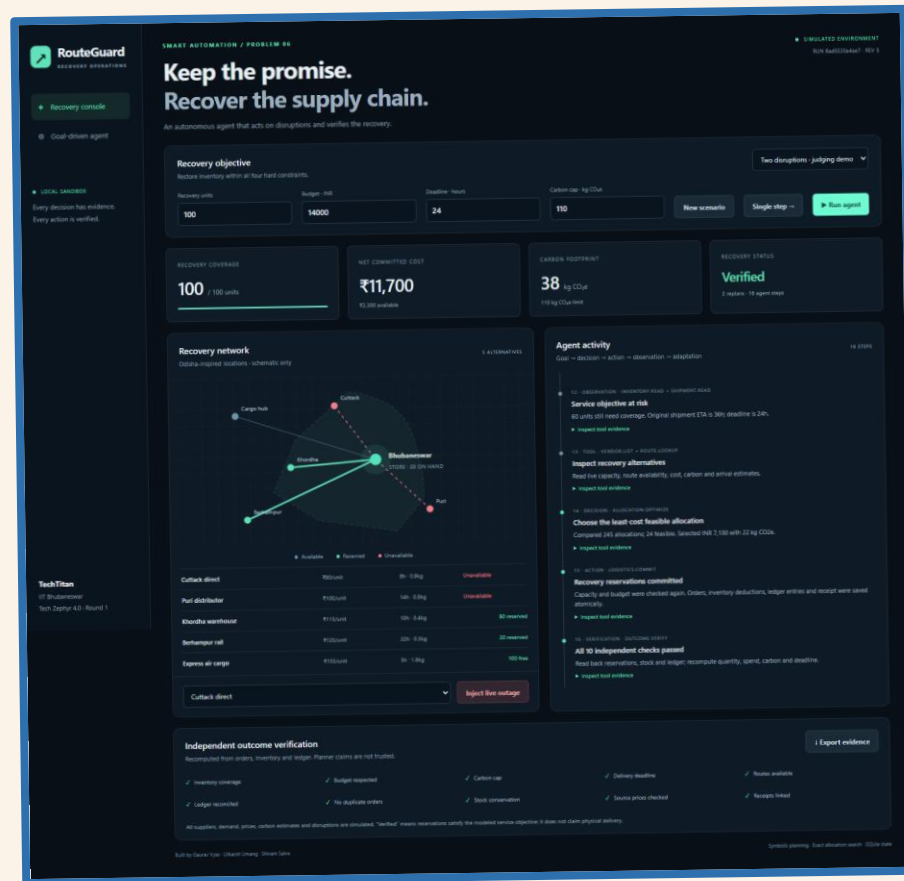
Exact feasibility and objective agreement

Cheapest-first baseline: feasible in 28 cases; constrained search: feasible in 31.

Limited five-offer sandbox benchmark. No production or generalization claim.

Working prototype

Inspect tool evidence, change constraints, inject an outage and export the full run.



python server.py

Open localhost:8765
Select Two disruptions
Run the agent
Inspect verification

Standard-library Python + browser

Challenges and limits

The prototype is designed to make recovery behavior inspectable and testable.

Consistent state after failure

Keep unaffected orders; cancel and refund affected ones; prevent duplicate charges.

Explicit simulator assumptions

One SKU, five offers, ten-unit lots, snapshot ETAs and fully refundable pre-dispatch cancellations. No live ERP, physical delivery or measured emissions.

Conclusion



+ A recovery action is complete only when the resulting state satisfies the goal.

**Act. Observe the consequence.
Replan. Verify.**

Delivered: runnable source, architecture, presentation,
demo video and reproducible evaluation evidence.

TechTitan · Gaurav Vyas · Utkarsh Umang · Shivam Salve

